

Chapter 5

Worksheet

Instructions:

Please complete all activities in this worksheet. Write your answers in the spaces provided or on separate sheets if needed.

Chapter 5: Debugging and Problem-Solving – Worksheet

Instructions: Complete all sections of this worksheet. For coding tasks, write or correct the code as instructed. For quick-response questions, provide brief answers. Use a separate sheet or notebook if needed for longer answers or code. This worksheet is designed to test your skills and help you learn from mistakes, so don't worry if you get stuck—ask for help or refer to Chapter 5!

Section 1: True/False Questions

Instructions: Read each statement carefully and mark whether it is True or False. (1 point each)

- **Debugging is the process of finding and fixing errors in a program.** ☐ True ☐ False
- **A syntax error occurs when the code runs but produces incorrect results due to flawed logic.** ☐ True ☐ False
- **A runtime error happens when the code crashes while executing, often due to invalid operations like dividing by zero.** ☐ True ☐ False
- **Using print statements is a debugging technique that helps track variable values during code execution.** ☐ True ☐ False
- **Edge case testing focuses on using only typical inputs to ensure the program works under normal conditions.** ☐ True ☐ False
- **Converting user input from a string to an integer is unnecessary when performing mathematical operations in Python.** ☐ True ☐ False

Section 2: Debugging Code Challenges

Instructions: Each task below provides a buggy code snippet. Identify the error(s), explain what's wrong, and write the corrected code. Test your corrected code with at least two different inputs to ensure it works. (3 points each)

Challenge 1: Snack Counter Bug

Buggy Code:

```
snacks = input("How many snacks sold? ")
total = snacks * 2.00 # Price per snack
print("Total money made: ", total)
```

- **Identify the Error(s):** What type of error is this, and why does it happen?
- **Corrected Code:** Write the fixed version below.

- **Test Inputs:** Use two different inputs (e.g., a valid number and an invalid input) and write the expected outputs.

Answer Space:

- Error(s): _____
- Corrected Code:

Write corrected code here

- Test Inputs and Outputs:
 - Input 1: _____ Expected Output: _____
 - Input 2: _____ Expected Output: _____

Challenge 2: Study Timer Logic Error

Buggy Code:

```
time_spent = int(input("Minutes studied: "))
if time_spent > 25: # Wrong condition
    print("Time for a break!")
else:
    print("Keep studying!")
```

- **Identify the Error(s):** What type of error is this, and why is it incorrect?
- **Corrected Code:** Write the fixed version below.
- **Test Inputs:** Use two different inputs and write the expected outputs.

Answer Space:

- Error(s): _____
- Corrected Code:

Write corrected code here

- Test Inputs and Outputs:
 - Input 1: _____ Expected Output: _____
 - Input 2: _____ Expected Output: _____

Section 3: Quick-Response Questions

Instructions: Answer the following questions briefly in 1-2 sentences. Be clear and specific. (2 points each)

- **Why are bugs a normal part of coding?**
- **What is the difference between a syntax error and a runtime error?**
- **How can “Rubber Duck Debugging” help when you’re stuck on a coding problem?**

Section 4: Creative Debugging Challenge

Instructions: Design a simple program idea related to something teens care about (e.g., a game score tracker, homework timer, or social media post counter). Write a short description of what the program does, then create a small piece of buggy code for it (with at least one intentional error). Finally, explain the bug and how to fix it. (5 points)

- **Program Idea Description:** What does your program do? (1-2 sentences)
- **Buggy Code:** Write a short code snippet (5-10 lines) with at least one intentional bug.

Write buggy code here

- **Bug Explanation and Fix:** What is the error, why does it happen, and how would you fix it?

Section 5: Reflection

Instructions: Reflect on what you've learned about debugging and problem-solving. Answer in 3-5 sentences. (3 points)

- **What was the most interesting or useful debugging technique you learned in this chapter, and why? How do you plan to use it in your future coding projects?**